Workflow Procedures

Written by Cody Hatfield

1. Issues

When a new feature is to be added, a new issue will be created to track development for that feature. Each issue can be considered a single unit of work, and should be doable by a single person (with only a few exceptions). You can view all issues by clicking on the "Issues" tab on the github page.

2. Creating Issues

To create a new issue, click on the green "New Issue" button within the Issues tab. When you create a new issue, you **must** include a special header at the front of the issue title. This header is formatted as [DCL-#], where the # sign is replaced with the issue number. The issue number will always be the number of open issues plus the number of closed issues at the moment the issue is created. If you type in the wrong number, you can change the title of the issue to match the number given on github at the end of the title line.

After you add the header, you will give a title to the issue that briefly describes what the issue is meant to accomplish. You will then give the issue a description which can range in size from a single of sentence to one or two paragraphs. If you have more details to add, you can add them in the comments of the issue.

Each issue will also need a label. You can assign a label by clicking on the "Labels" button and then clicking on one of the label options. The available labels are:

- documentation
  - Documentation files such as this one are being written
- analysis
  - A complex problem will be broken down and likely split into multiple issues. May or may not commit code.
- bug
  - A bug was discovered in the code, and need to be fixed.
- cancelled
  - An issue was cancelled for any reason
- complete
  - An issue is complete and thus closed.
- feature
  - A new feature that will be added by changing code or a configuration

Finally, you must assign the issue to the [DCL] Drone Combat Lab project. This can be done by clicking on "Projects" button and then selecting the DCL from the pop-up menu.

3. Workflow

We will use a basic Kanban board to organize our issues. This board can be viewed by going onto the github page and clicking Projects > [DCL] Drone Combat Lab. Issues on this board can be dragged and dropped in real time.

Issues can exist in one of the following columns:

- Todo
  - The issue needs to be done but isn't assigned to anyone.
- In Progress
  - The issue is currently being worked on by someone.
- Quality Assurance
  - The issue needs to go through quality assurance.
- Done
  - The issue is completed and closed.
- Cancelled
  - The issue was cancelled and closed.

When a new issue is created, it is added to the Todo column by default. When a person assigns themself to an issue, they move that issue to the In Progress column. Once an issue is completed, the developer moves it to the Quality Assurance column. Issues in Quality Assurance can only be QA'd by someone who did **not** develop code on the issue. Once the issue passes QA, it is moved to the Done column. The cancelled column is only used if an issue needs to be cancelled, i.e. duplicate issue or the feature is decided to no longer be needed.

4. Commit Message Formatting

When you commit code to the repository, you must always add a commit message too. This can be specified in the commit command by adding a -m argument like so:

- git commit -m "Commit message"

Each commit message must follow a very specified formatting. The message **must** begin with the issue header that this code commit is associated with, such as [DCL-29] or [DCL-20]. Once you have added the header, you must then follow it with a short message (no longer than two or three sentences, no newlines) that describes roughly what was done in the commit. There may be multiple commits per issue, as such it is recommended that you specify what you actually did in the commit rather than just copy the issue title. An example commit message with command is:

- git commit -m "[DCL-29] A new document about workflow has been created"

5. Continuous Integration

Continuous Integration (CI) is a workflow similar to Kanban, however it comes with some interesting development practices that I have found very useful in my career. Chief among these is the usage of a **single** development branch. Unlike traditional Agile workflows, which often have each developer create their own branch off a main branch that is eventually merged back into, all developers will be working on and committing to the same branch simultaneously.

We will be taking this branch philosophy of CI and using it in our Kanban workflow. Thus, our repo will only contain two types of branches:

- dev
    - The branch where active development is taking place.
- release
    - A branch which provides a stable release of the code and a specified list of basic features that differentiates it from previous releases.

There will only be one dev branch, but there may be many release branches. All work during active development should be committed directly to the dev branch. No new code should be committed to a release branch – if a release branch needs to be updated, then it should be merged with the dev branch to receive these updates.

While this simultaneous work may feel counterintuitive, it does offer a very important benefit – all developers will be forced to rectify merge conflicts at the exact moment they try to push new code to the repo. This will prevent merge conflicts from piling up and becoming ever more complex as more and more development branches are added. I worked on a large team that made the switch from a very traditional Agile flow to a slimmed down Continuous Integration approach, so believe me when I say that the efficiency difference between the two is like night and day.

6. Program Formatting and Self-Documenting Code

We will implement some program formatting requirements as a part of our workflow. These requirements may seem strict at first, but they are necessary to keep our codebase standardized, especially given the wild-west nature of python and its wide plethora of coding styles.

Of particular importance is for us to create code that is **self-documenting.** While it is true that our team should do its best to create documentation for the DCL upon its completion, it is also true that the quickest way for someone to understand your code is to write it so that it is easy to read and comprehend. A lot of our program formatting rules are written with this purpose in mind, as code that is easy to read is also easy to debug and easy to update.

6a. No Comments

Documentation is important, but it should be kept out of the code files. There should be **no comments** in your code. The only exception to this rule possible using comments to describe options in a config file, but otherwise your code should not have any comments in it. Not only do comments clutter up visual space and increase the cognitive strain on the reader, but removing them will force you to write better code that you can more easily understand.

6b. One Class, One File

Each file should contain exactly one class, and nothing else. There may be very, very special cases where a global function is needed, but these cases will be **extremely** rare and not considered standard procedure. Outside of these special cases, you should consider this project to be fully object oriented.

6c. Interface Over Inheritance

Inheritance is considered one of the biggest innovations of Object Oriented Programming (OOP). However, as years passed since its creation, skepticism has been raised about how effective it actually is

as a programming technique. Especially when one is dealing with large and complex OOP projects, the sheer complexity of resolving inheritance trees grows exponentially with each layer added.

Thus, you should follow the Interface design pattern rather than the Inheritance design pattern. With Inheritance, you would abstract shared functions between classes into a shared class from which they both inherit. With the Interface pattern, you don't abstract the *whole* function, instead you abstract the function header and **only** the function header to a shared Interface that both classes implement. This requires that the body of the function be implemented individually in each class.

While Interface does lead to a lot of code duplication, it is also much more stable and flexible. Furthermore, you no longer need to work up an inheritance tree in order to determine what functionality is possible for a group of classes – by looking at their single shared Interface, you can understand exactly what these classes are capable of and how they are used.

A helpful way to understand Interfaces is to view them as a **contract** that both classes have signed. While each may do their shared actions in different ways, both classes are guaranteed to be able to perform those actions.

Python does not have native support for Interfaces. As such, we will implement ours as classes, and simply raise a NotImplementedError() under each function that an implementing class is required to define. You can view examples of this in the code base.

Keep in mind that Inheritance is **not** outright forbidden in the same way comments are. There may be some instances where Inheritance makes the most sense and should be used. However there should only be a handful of these instances among the hundreds of classes which created in this project. Unless something absolutely need to be done through Inheritance, then it should be done through Interface.

6d. Capitalization, Camel-Case, and Naming

There is a need to control Capitalization to help keep the program look/feel unified and thus easy to read.

The following items should have their first letter be **uppercase** and the name be **camel-case**:

- Name of a class's file
  - DynamicObject.py
- Name of a class
  - DynamicObject
- Name of a function
  - GetPosition(self)

The following items should have their first letter be **lowercase** and the name be **underscore-case**:

- Name of a folder
  - entities
- Name of a file (except a class's file)
  - log_data.txt
- Name of a variable
  - self.entity_logger

Names of variables, functions, and classes should be as descriptive as possible. It is better to spend 3 seconds extra typing out a long name if it means saving yourself 3 hours of debugging in the future.

6e. Function Length

Functions should in general be no longer than 100 lines, although 50 should be more of the average you're shooting for. If a function is too long, then it is hard to understand, and it should be split up into sub-functions and possibly other classes.

Sometimes when calling a function, the arguments given will be too long horizontally. If this is the case, then put each argument onto a newline such that a nesting effect is provided.